

گزارش پروژه ی دوم سیستم های توزیع شده

محمد امین توانایی (۸۱۰۱۰۱۳۹۶) – مجید صادقی نژاد (۸۱۰۱۰۱۴۵۹)

بهار ۱۴۰۵

مقدمه و معماری کلی سیستم

هدف اصلی در این پروژه، طراحی و پیاده سازی یک سیستم توزیع شده با استفاده از معماری میکروسرویس (Microservices) بود تا نشان دهیم چگونه سرویس های مختلف می توانند در محیط های کاملاً ایزوله از یکدیگر کار کنند و از طریق فراخوانی های شبکه ای با هم در ارتباط باشند. کل این سیستم بر روی ماشین های مجازی مجزا سوار شده است تا محیط اجرای هر بخش کاملاً مستقل باشد. برای شبیه سازی این محیط در سیستم عامل خودم، از ابزار Multipass استفاده کردم که به سرعت ماشین های مجازی سبک مبتنی بر اوبونتو را اجرا می کند. مزیت این معماری این است که اگر به فرض مثال سرور وب ما مورد نفوذ قرار بگیرد، هکر به فایل های سیستمی یا پایگاه داده کاربران دسترسی مستقیم نخواهد داشت زیرا آن داده ها در ماشین های مجازی دیگری قرار دارند و تنها از طریق توابع از پیش تعریف شده و محدود می توان به آن ها درخواست فرستاد.

نقش هر ماشین مجازی

در این معماری، من چهار ماشین مجازی (VM) مجزا را راه اندازی و استفاده کردم که هر کدام وظیفه بسیار مشخص و تک منظوره ای را ایفا می کنند.

ماشین مجازی اول (VM1) یا همان سرویس وب، خط مقدم سیستم ما است. این ماشین مستقیماً با کاربر نهایی از طریق مرورگر در ارتباط است و روی پورت ۸۰۰۰ به درخواست های HTTP پاسخ می دهد. هیچ دیتابیس یا فایل حساسی روی این ماشین قرار ندارد و وظیفه اصلی آن گرفتن اطلاعات از صفحات HTML و هدایت آن ها به سمت سایر ماشین ها است.

ماشین مجازی دوم (VM2) وظیفه خطیر احراز هویت را بر دوش دارد. داده های مرتبط با کاربران سیستم (همانند فایل کانفیگ JSON که شامل نام کاربری و رمزهای عبور است) منحصراً درون این ماشین ذخیره شده اند. برای اینکه امنیت در بالاترین سطح ممکن باشد، حتی در داخل این ماشین نیز رمزهای عبور به صورت متن ساده (Plain Text) ذخیره نشده اند، بلکه از الگوریتم رمزنگاری یک طرفه bcrypt برای هش کردن آن ها استفاده کرده ام.

ماشین مجازی سوم (VM3) نقش یک سرور فایل یا File Repository تاریک را بازی می کند. این یعنی مستقیماً به اینترنت متصل نیست و هیچ وب سروری ندارد. فایل های حساس سیستم و

تصاویر فقط درون این ماشین مجازی نگهداری می‌شوند و تنها از طریق پروتکل‌های داخلی شبکه خوانده می‌شوند.

ماشین مجازی چهارم (Node-4) نیز اختصاصاً برای پیاده‌سازی سیستم مانیتورینگ و گوش دادن به رویدادهای سیستمی (Subscriber) در نظر گرفته شده است که جلوتر به صورت مفصل در بخش Pub/Sub به آن خواهیم پرداخت.

آدرس IP ماشین‌های مجازی

برای اینکه این ماشین‌ها بتوانند همدیگر را پیدا کنند، از آی‌پی‌های اختصاص داده شده توسط شبکه داخلی Multipass استفاده کردم. برای جلوگیری از هاردکد شدن (Hardcode) آی‌پی‌ها در سورس‌کد، آن‌ها را به عنوان متغیرهای محیطی (Environment Variables) به سیستم دادم تا اگر آی‌پی ماشینی در راه‌اندازی مجدد تغییر کرد، نیازی به کامپایل دوباره کدهای زبان Go نباشد.

شات آی‌پی‌های تخصیص یافته به ماشین‌ها را می‌توانید در زیر مشاهده کنید:

~ # multipass list				majidsadeghinejad@Macy	
Name	State	IPv4	Image		
node-1	Running	192.168.252.6	Ubuntu 26.04 LTS		
node-2	Running	192.168.252.7	Ubuntu 26.04 LTS		
node-3	Running	192.168.252.8	Ubuntu 26.04 LTS		
node-4	Running	192.168.252.9	Ubuntu 26.04 LTS		

فناوری انتخاب شده برای RPC و دلایل آن

برای ارتباط بین ماشین وب با ماشین‌های فایل و احراز هویت، از استفاده از API‌های معمول RESTful خودداری کردم و به جای آن از تکنولوژی RPC استاندارد تعبیه شده در هسته زبان Go یعنی پکیج `net/rpc` استفاده کردم. این ارتباط بر بستر پروتکل پروتکل خام TCP برقرار می‌شود.

دلیل اصلی من برای این انتخاب، سادگی و پرفورمنس بسیار بالا در محیط‌هایی است که کلاینت و سرور هر دو با یک زبان (در اینجا زبان Go) نوشته شده‌اند. اگر چه امروزه فریم‌ورک‌هایی مثل gRPC بسیار محبوب هستند، اما استفاده از gRPC نیازمند نصب ابزارهای جانبی برای کامپایل فایل‌های `proto` و نوشتن کدهای میانی زیاد است. در مقابل، پکیج `net/rpc` به من اجازه داد تا با تعریف کردن Struct های ساده در Go، به سرعت اشیا را از طریق شبکه (Serialization) توسط فرمت باینری gob (به صورت پیش‌فرض) ارسال و دریافت کنم که به شدت سربار (Overhead) سیستم را کاهش داد و عملکرد شبکه را بهینه کرد.

توضیح Procedure های پیاده‌سازی شده در سیستم

روند فراخوانی از راه دور را از طریق تعریف دو پروسیجر اصلی در سیستم جلو بردم:

روند اول `AuthService.Login`` است. وقتی کاربر در مرورگر خود فرم لاگین را ثبت می‌کند، ماشین اول این تابع را روی ماشین دوم با دو پارامتر `Username` و `Password` فراخوانی می‌کند. در داخل این تابع در سرور احراز هویت، ابتدا بررسی می‌شود که آیا کاربر در حافظه وجود دارد یا خیر. اگر وجود داشت، پسورد ارسال شده با استفاده از تابع `bcrypt.CompareHashAndPassword`` با پسورد هش شده در سیستم مطابقت داده می‌شود و در نهایت یک `Struct` شامل یک متغیر بولین به اسم `Success`` برای ماشین اول بازگردانده می‌شود تا وب‌سرور بداند باید کاربر را به صفحه اصلی هدایت کند یا خطای ورود نشان دهد.

روند دوم مربوط به `FileService.GetFile`` است. این پروسیجر در ماشین سوم قرار دارد. این تابع نام فولدر و نام فایل را به عنوان ورودی دریافت می‌کند. یکی از نکات مهم امنیتی که در این تابع پیاده کردم، بحث ایمن‌سازی مسیر (Sanitization) بود تا در برابر حملات `Path Traversal` مقاوم باشد. این تابع ابتدا چک می‌کند که دایرکتوری درخواستی فقط از فولدرهای مجاز (مثل `images`) باشد. سپس با استفاده از `os.ReadFile`` فایل را از هارددیسک ماشین سوم می‌خواند و آن را در یک آرایه باینری (`Byte Slice`) در ریپلای `RPC` به سرویس وب باز می‌گرداند.

نحوه دریافت فایل از VM3

وقتی کاربر با موفقیت لاگین می‌کند و کوکی احراز هویت برای او ثبت می‌شود، مرورگر او تلاش می‌کند که عکس موجود در صفحه را لود کند. سورس این عکس به آدرس `/fetch-image`` در ماشین وب (`VM1`) اشاره می‌کند. وقتی این درخواست به وب‌سرور می‌رسد، وب‌سرور ابتدا کوکی‌ها را چک می‌کند تا مطمئن شود فقط کاربران تایید شده حق دریافت فایل را دارند. اگر کاربر لاگین بود، وب‌سرور به عنوان کلاینت به ماشین سوم متصل می‌شود و پروسیجر دریافت فایل را صدا می‌زند. ماشین سوم بایت‌های فایل لوگو را از طریق شبکه `TCP` به ماشین اول پاس می‌دهد. سپس ماشین اول، نوع محتوا در هدر `HTTP` را برابر با `image/png`` قرار داده و مستقیم بایت‌ها را در بدنه ریسپانس (`Response Body`) برای مرورگر چاپ می‌کند. بنابراین کاربر بدون اینکه بداند فایل از کجا آمده، تصویر را می‌بیند.

نحوه ی ارتباط VM1 و VM2:

یکی از مهم‌ترین تفاوت‌های ارتباطات مبتنی بر RPC با استفاده از REST HTTP کلاسیک در این است که در REST، ارتباطات به صورت استاندارد حول موجودیت‌ها (منابع) و با استفاده از افعال اصلی HTTP (نظیر GET، POST و PUT) تعریف می‌شود؛ اما در یک پیاده‌سازی RPC-Like هر درخواست در واقع درخواستی برای اجرای یک تابع خاص در سرور مقابل با پارامترهای مشخص است، حتی اگر برای انتقال این درخواست از بستر HTTP استفاده شود. بنابراین در REST بر روی "چه چیزی" (منابع) تمرکز داریم اما در RPC تمرکز روی "چه کاری انجام شود" (توابع) است.

با در نظر گرفتن این موارد و برای دستیابی به بالاترین میزان یکپارچگی، کارایی و کسب امتیاز طراحی بهتر، در این پروژه به جای استفاده از REST یا شبیه‌سازی‌های مبتنی بر HTTP، از کتابخانه استاندارد net/rpc مختص زبان Go استفاده شده است. این پیاده‌سازی یک RPC کاملاً واقعی (True RPC) به حساب می‌آید. برخلاف متدهای RPC-Like که سربار پردازش هدرهای HTTP و تبدیل داده‌ها به JSON را به شبکه تحمیل می‌کنند، سیستم طراحی شده‌ی ما داده‌ها را از طریق فرمت دودویی و کم‌حجم gob (فرمت داخلی Serialization گو برای انتقال داده‌ها بین سرورهای هم‌زبان) و مستقیماً روی بستر سوکت‌های خام شبکه (نشست‌های TCP اتصال به پورت) مبادله می‌کند. با وجود اینکه ابزارهای پیچیده‌تری مانند gRPC نیز گزینه‌های مقبولى به شمار می‌روند، net/rpc به من اجازه داد که با کمترین میزان پیچیدگی تنظیمات و عدم نیاز به راه‌اندازی فایل‌های Proto، پروسیجرهای احراز هویت و دسترسی به فایل‌ها (مانند AuthService.Login و FileService.GetFiles) را با سرعت بی‌نظیری در محیط سیستم‌عامل لینوکس ماشین‌های مجازی فراخوانی و اجرا نمایم.

روش پایش حافظه و رفع نشت (Memory Monitoring)

یکی از الزامات یک سیستم توزیع‌شده پایدار، اطلاع از وضعیت سلامت نودها است. برای این کار من یک ساختار پایش حافظه در پس‌زمینه ماشین وب ایجاد کردم. در تابع مجزایی به نام monitorMemory که به صورت یک Goroutine در پس‌زمینه (همزمان با بالا آمدن سرور اصلی) اجرا می‌شود، از یک time.Ticker استفاده کردم تا هر ۵ ثانیه، وضعیت حافظه Garbage Collector سیستم با متد runtime.ReadMemStats خوانده شود.

برای اینکه بتوانم سیستم را تست کنم، یک روت مخفی به اسم consume-memory / نوشتم که با هر بار فراخوانی درخواست کاربر را پردازش کرده و بلوک‌های بزرگ چند مگابایتی (به میزان دلخواه) ایجاد کرده و در یک متغیر سراسری نگه می‌دارد (شبیه‌سازی نشت حافظه یا Memory Leak) وقتی

حافظه اشغال شده اکتیو سرور از آستانه مشخصی (مثلا ۳۰۰ مگابایت) فراتر می‌رود، پروسه پایش حافظه وارد عمل شده و روال Pub/Sub را استارت می‌زند.

توضیح سازوکار Pub/Sub

من الگوی Pub/Sub را به صورت یک Webhook توسعه دادم. در زمان بالا رفتن مصرف حافظه (همانطور که در بخش قبل گفته شد)، ماشین اول ما نقش (Publisher) را بر عهده می‌گیرد. ماشین اول یک Struct با نام `MemoryEvent` می‌سازد که شامل نوع رخداد، اسم سرویسی که دچار مشکل شده، میزان حافظه لورفته و مهر زمانی (Timestamp) است. این اطلاعات به فرمت JSON درآمده و از طریق پروتکل HTTP به ماشین چهارم (Node-4) که نقش (Subscriber) را دارد ارسال (Post) می‌شود. ماشین مشترک یک سرور مستقل است که همیشه روی پورت ۹۰۰۰ در حال گوش دادن است و پس از دریافت پیام، اگر متوجه شود که هشدار از نوع `HIGH\_MEMORY\_USAGE` است، یک آگاهی امنیتی بزرگ با جزئیات کامل روی کنسول خود چاپ می‌کند تا مهندسین سیستم بتوانند سریعاً به داد سرور در حال کرش برسند!

مشکلات و محدودیت‌های پیاده‌سازی

اولین مشکل، بحث معماری پردازنده‌ها بود. سیستم عامل مورد استفاده من مبتنی بر معماری ARM بود بنابراین بیلد اولیه برنامه‌ها روی ماشین‌های مجازی لینوکسی خطا می‌داد. با بررسی متوجه شدم باید قابلیت Cross-Compilation زبان گو را فعال کنم تا برای سیستم هدف باینری بسازد؛ از این رو کامپایل کدها را با فلگ‌های `GOOS=linux` و `GOARCH=arm64` انجام دادم که مشکل را به کلی رفع کرد.

مشکل دوم در فرایند ساخت Publisher/Subscriber پیش آمد. زمانی که تلاش می‌کردم باینری سابسکریبر که با موفقیت کامپایل شده بود را روی ماشین چهارم اجرا کنم با خطای عجیب `syntax error near unexpected token '<arch>!'` روبرو شدم و اجازه اجرای فایل در لینوکس داده نمی‌شد. حتی با دستورات تغییر پرمیشن `chmod` نیز این مشکل برقرار بود. پس از دیباگ متوجه شدم که در فایل منبع، نام پکیج به جای `package main` به اشتباه `package subscriber` گذاشته شده بود. در محیط زبان Go، فقط و فقط پکیج‌های main توانایی تبدیل شدن به فایل اجرایی (Executable) را دارند و سایر پکیج‌ها صرفاً به فایل‌های آرشیو (از فرمت a) کامپایل می‌شوند که برای کتابخانه‌ها استفاده می‌شوند و همین موضوع جلوی اجرای روی لینوکس را می‌گرفت. پس از تصحیح هدر به `package main` این مشکل نیز برطرف شد.

همچنین چالش دیگری با ابزار Multipass داشتیم که اجازه انتقال (Transfer) مستقیم پوشه‌ها به شکل ریکرسیو با پرچم `R` را در محیط مک نمی‌داد که مجبور شدم فولدرهای تصاویر و تمپلیت‌ها را به صورت دستی روی ماشین‌ها بسازم و فایل‌ها را تک‌تک وارد نمایم.

اجرای موفقیت آمیز سرویس‌ها (سناریو ی تست)

نمای اجرای سرویس احراز هویت (VM2):

```
ubuntu@node-2:~$ ./auth-server
2026/06/01 04:39:14 Starting Auth Service (VM2)...
2026/06/01 04:39:14 Loaded 3 users into memory.
2026/06/01 04:39:14 Auth RPC Service listening on port 8002...
```

نمای اجرای سرویس فایل (VM3):

```
ubuntu@node-3:~/file-vm$ ./file-server
2026/06/01 04:42:43 Starting File Server (VM3)...
2026/06/01 04:42:43 File RPC Service listening on port 8003...
```

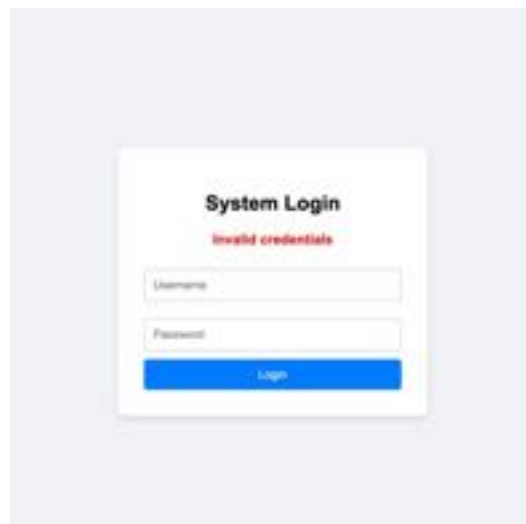
نمای اجرای وب سرور و ثبت رویداد لاگین (VM1):

```
ubuntu@node-1:~/web-vm$ export AUTH_VM_IP="192.168.252.7:8002"
ubuntu@node-1:~/web-vm$ export FILE_VM_IP="192.168.252.8:8003"
ubuntu@node-1:~/web-vm$ export SUBSCRIBER_IP="192.168.252.9:9000"
ubuntu@node-1:~/web-vm$ ./web-server
2026/06/01 04:50:49 Starting Web Service (VM1) on port 8000...
```

نمای مرورگر از صفحه دسترسی موفق سیستم توزیع شده:



ورود اطلاعات نادرست:



ورود اطلاعات درست و نمایش صفحه ی خوش آمد به همراه دریافت عکس از فایل سرور:



راه اندازی ماشین مجازی: subscriber

